Blending ensemble for classification (ML)

Pranav Patil, CQF June 23

Abstract

Stock trend prediction using machine-learning algorithms aims at developing effective and efficient models that can provide a better and higher rate of prediction accuracy. The problem statement is to produce a model to predict positive moves (up trend) using the Blending Ensemble Technique. Four ensemble classifiers have been applied in stock trend predictions, which are Decision Tree Classifier, Random Forest Classifier, KNN Classifier and XGBoost Classifier. In this study, I stacked these four classifiers into a stacked classifier which works as a stacked ensemble with Logistic regressor as an final estimator and analysis of each base model along with the meta model. The study outcome shows that stacking and blending ensemble technique offer higher prediction accuracies (90–100%). Finally, the results undoubtedly suggest that an innovative study in the domain of stock market direction prediction ought to include ensemble techniques in their sets of algorithms.

***Keywords*:** Machine Learning, Ensemble Learning, Decision Tree Classifier, Random Forest Classifier, KNN Classifier, XGBoost Classifier, Stacked Classifier, Logistic Regression, Prediction, Accuracy.

I. Introduction

Stock trend prediction is very valuable for investment management. Accurate prediction makes it possible for investors to decide a proper moment to buy or sell a stock, to achieve the goal to beat the market and make profits. However, stock trend prediction is really challenging due to the high volatility in the stock market. The trend prediction in stock market has drawn a lot of research attention for many decades using both statistical and computing approaches including artificial intelligence. This work aims to apply four well-known classification algorithms, namely, Decision Tree Classifier, Random Forest Classifier, KNN Classifier, Extreme Gradient Boosting (XGBoost) for trend prediction. The aim is to propose a heterogeneous ensemble classifier from such four base algorithms to predict trend of stock price for trading recommendation. Performance of the algorithms will be evaluated with the investment returns and risks from trading simulation Specifically, in the first step, the classifiers are trained to build a trend prediction model from a daily dataset. The experiment uses publicly available data of open, high, low, close, and volume of stock prices using the predicted trend results.

The computed technical indicators are fed into the process of building the prediction models. However, up to now, there is no known set of technical indicators most suitable for a dataset.

A feature (or attribute) selection method is adopted in the data preprocessing using Exploratory Data Analysis of which Correlation Analysis & Multicollinearity Analysis plays crucial role to choose the optimal technical indicators as model inputs. The models are used for predicting daily trends of the corresponding testing (or unseen) data, which will further be used for trading simulation in the second step. Moreover, choosing the optimal classification algorithm and its hyperparameter for a given data often requires expertise and amount of effort. Hyperparameters are parameters of the algorithm itself that need to be set to suitable values to obtain good performance for a given data. Most of the time they are left to using default values, which unfortunately often lead to performance much lower than of the hyperparameter tuning. Thus in this work, performance and optimum parameters for a given algorithm are explored by using grid search method.

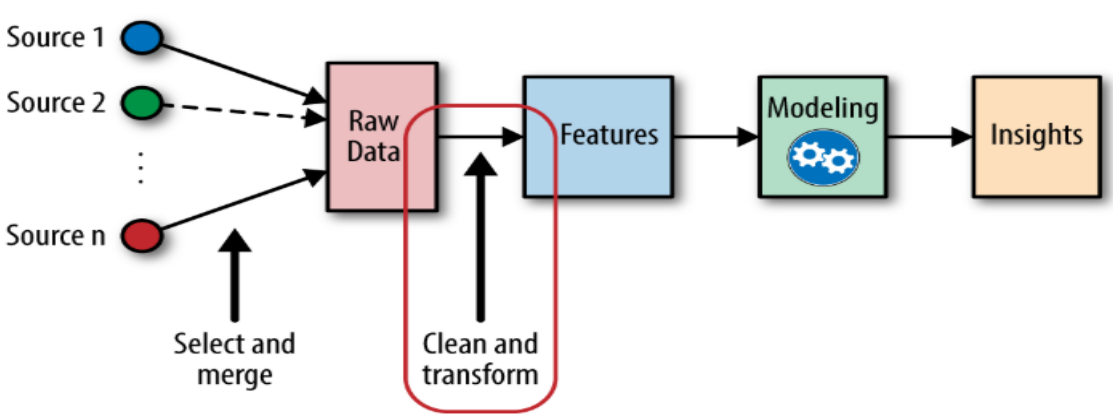
Finally, the predicted trend results are employed in a trading recommendation system – that is to buy, to sell, or to do nothing if the predicted trend is positive, negative, or very small, respectively. The investment returns from trading simulation with the testing data are calculated. Performance of the proposed model is evaluated using trading return and Sharpe ratio. Sharpe ratio is a measure of risk-adjusted return, describing how much excess return received for the volatility (or risk) of holding a riskier asset.

II. Dataset and Features Engineering

For this study, I have used historical dataset of the ticker 'TSLA', over the time horizon of five years, in 'OHLCV' format. The computed technical indicators, features which are fed into the process of building the prediction models, are :

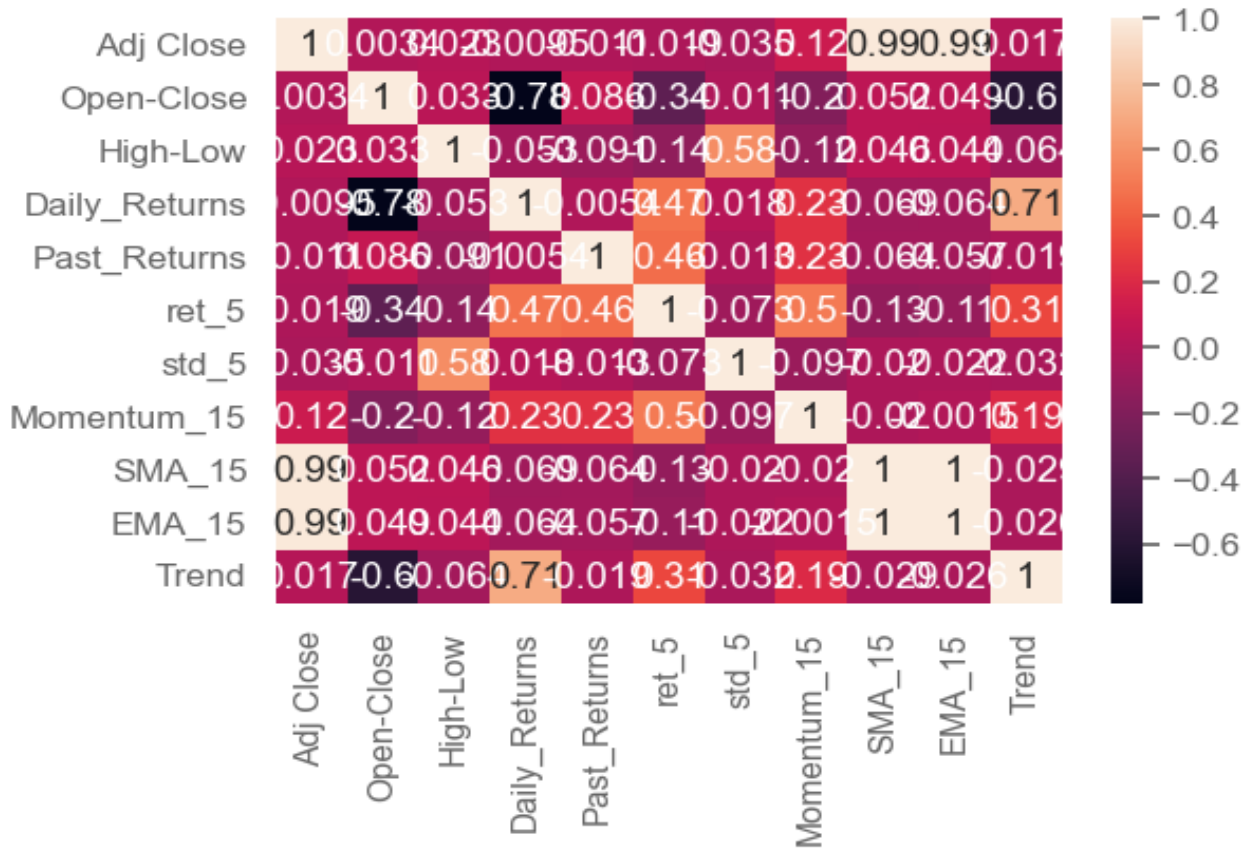
```
# Features construction
data['Open-Close'] = (data.Open - data.Close)/data.Open
data['High-Low'] = (data.High - data.Low)/data.Low
data['Daily_Returns'] = np.log(data['Close']/data['Close'].shift(1))
data['Past_Returns'] = data['Daily_Returns'].shift(1)
data['ret_5'] = data['Daily_Returns'].rolling(5).mean()
data['std_5'] = data['Daily_Returns'].rolling(5).std()
data['Momentum_15'] = data['Close'] - data['Close'].shift(15)
data['SMA_15'] = data['Close'].rolling(window=15).mean()
data['EMA_15'] = data['Close'].ewm(span=15,min_periods=15).mean()
data['Trend'] = np.where(data['Daily_Returns'] > 0, 1, 0)
```

Feature Engineering is the process of extracting and organizing the important features from raw data in such a way that it fits the purpose of the machine learning model. It can be thought of as the art of selecting the important features and transforming them into refined and meaningful features that suit the model.



Feature Engineering encapsulates various data engineering techniques such as selecting relevant features, handling missing data, encoding the data, and normalizing it. It is one of the most crucial tasks and plays a major role in determining the outcome of a model. In order to ensure that the chosen algorithm can perform to its optimum capability, it is important to engineer the features of the input data effectively.

In this study, features engineering and transformation has been done using ***Exploratory Data Analysis***, particularly by ***Correlation Analysis*** & ***Multicollinearity Analysis***. Correlation among the features can be studied using ***correlation matrix*** of the dataset and highly correlated features can be omitted.



Multicollinearity occurs when two or more independent variables in a data frame have a high correlation with one another in a regression model.

In the case of multicollinearity analysis of the features, Detecting multicollinearity is important because while multicollinearity does not reduce the explanatory power of the model, it does reduce the statistical significance of the independent variables.

We can detect the multicollinearity among the features using the ***Variance Inflation Factor (VIF).***

A variance inflation factor (VIF) is a measure of the amount of multicollinearity in regression analysis. Multicollinearity exists when there is a correlation between multiple independent variables in a multiple regression model. This can adversely affect the regression results. Thus, the variance inflation factor can estimate how much the variance of a regression coefficient is inflated due to multicollinearity.

It is computed by,

$$VIF_i = \frac{1}{1 - R_i^2}$$

where R_i^2 is the unadjusted coefficient for regressing the *i-th* independent variable on the remaining ones. When it is is equal to 0, and therefore, when VIF or tolerance is equal to 1, the *i-th* independent variable is not correlated to the remaining ones, meaning that multicollinearity does not exist.

In general terms,

- VIF equal to 1 implies that the variables are not correlated.
- VIF between 1 and 5 implies that the variables are moderately correlated.
- VIF greater than 5 implies that the variables are highly correlated.

The higher the VIF, the higher the possibility that multicollinearity exists, and further research is required. When VIF is higher than 10, there is significant multicollinearity that needs to be corrected.

VIF values of the features are,

	Features	VIF
0	Adj Close	1787.780778
1	Open-Close	2.663716
2	High-Low	5.604550
3	Daily_Returns	3.105358
4	Past_Returns	1.473213
5	ret_5	2.666660
6	std_5	5.303470
7	Momentum_15	5.450348
8	SMA_15	4621.554152
9	EMA_15	8007.464979

We can see here that the ‘Adj Close’ ,‘SMA_15’ & 'EMA_15' have a very high VIF value, meaning they can be predicted by other features in the dataset. VIF exceeding 5 or 10 indicates high multicollinearity between the features.

Dropping these highly correlated features will help in bringing down the multicollinearity between other correlated features.

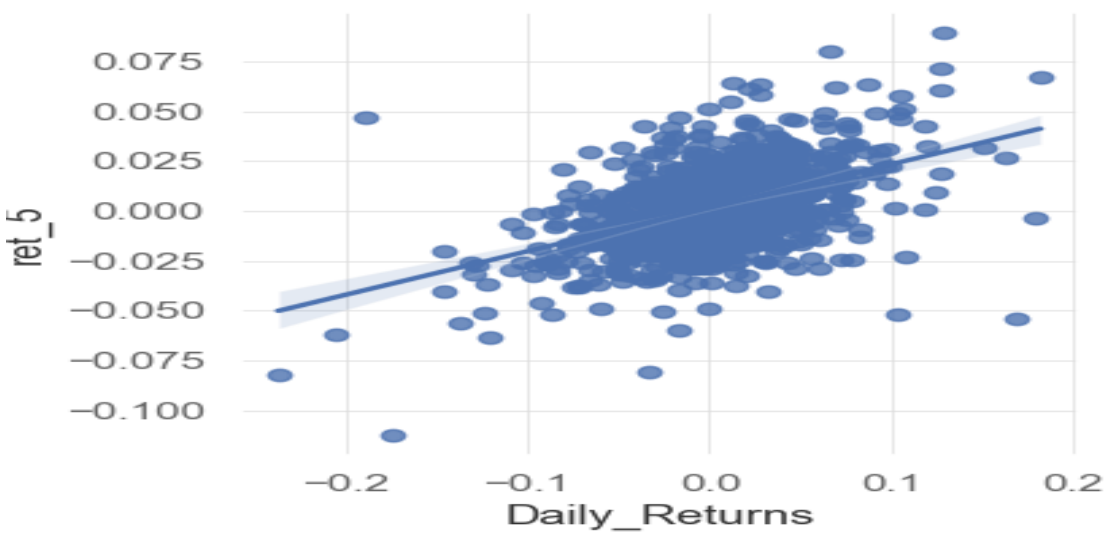
```
df = data.drop(['Adj Close','High-Low','std_5','Momentum_15','SMA_15','EMA_15'],axis=1)
calc_vif(df)
```

	Features	VIF
0	Open-Close	2.633257
1	Daily_Returns	3.241903
2	Past_Returns	1.396606
3	ret_5	1.779369
4	Trend	1.373394

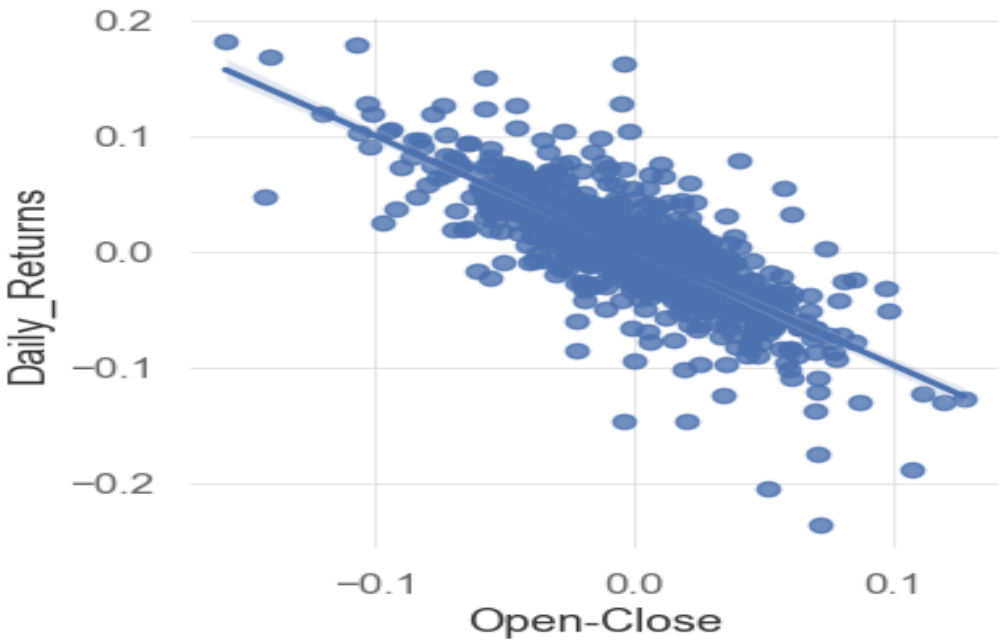
Now, We can use these selected features as input and target variables in the models for training and testing.

Relationship among the these features can be visualized using regression plots:

1. *Daily Returns vs Ret_5 (five days returns):*



2. *'Open-Close' vs 'Daily_Returns'*:

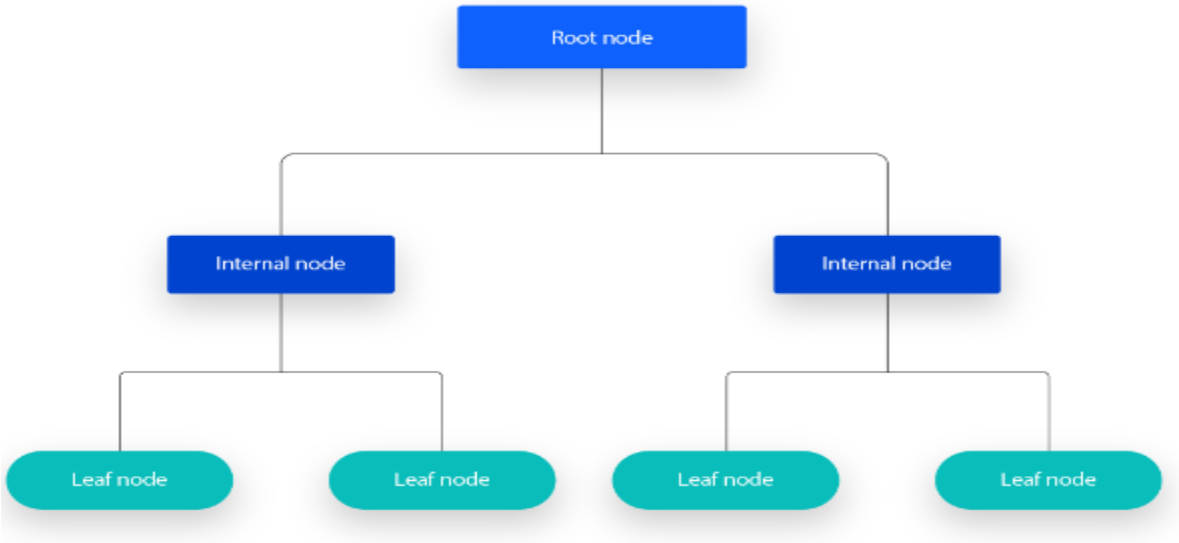


3. *Past Returns vs 'Ret_5' (five days returns):*

III. Base Models

1. **Decision Tree Classifier**:

A decision tree is a non-parametric supervised learning algorithm, which is utilized for both classification and regression tasks. It has a hierarchical, tree structure, which consists of a root node, branches, internal nodes and leaf nodes.



As you can see from the diagram above, a decision tree starts with a root node, which does not have any incoming branches. The outgoing branches from the root node then feed into the internal nodes, also known as decision nodes. Based on the available features, both node types conduct evaluations to form homogenous subsets, which are denoted by leaf nodes, or terminal nodes. The leaf nodes represent all the possible outcomes within the dataset.

Decision tree learning employs a divide and conquer strategy by conducting a greedy search to identify the optimal split points within a tree. This process of splitting is then repeated in a top-down, recursive manner until all, or the majority of records have been classified under specific class labels. Whether or not all data points are classified as homogenous sets is largely dependent on the complexity of the decision tree. Smaller trees are more easily able to attain pure leaf nodes—i.e. data points in a single class. However, as a tree grows in size, it becomes increasingly difficult to maintain this purity, and it usually results in too little data falling within a given subtree. When this occurs, it is known as data fragmentation, and it can often lead to overfitting. In data mining, decision trees can be described also as the combination of mathematical and computational techniques to aid the description, categorization and generalization of a given set of data.

Data comes in records of the form:

$$(x, Y) = (x_1, x_2, x_3, \dots, x_k, Y)$$

The dependent variable,Y, is the target variable that we are trying to understand to classify or generalize. The vector x is composed of the features, x_1,x_2,x_3 etc., that are used for that task.

In the case of classification, Classification tree analysis is performed when the predicted outcome is the class (discrete) to which the data belongs.

While there are multiple ways to select the best attribute at each node, two methods, Information gain and Gini impurity, act as popular splitting criterion for decision tree models. They help to evaluate the quality of each test condition and how well it will be able to classify samples into a class.

Entropy and Information Gain:

It's difficult to explain information gain without first discussing entropy. Entropy is a concept that stems from information theory, which measures the impurity of the sample values.

Entropy is defined with by the following equation,

$$Entropy(S) = -\sum_{c \in C} p(c).log_2 p(c)$$

where:

1. *S* represents the data set that entropy is calculated .
2. *c* represents the classes in set, *S*.
3. *p(c)* represents the proportion of data points that belong to class *c* to the number of total data points in set, *S*.

Entropy values can fall between 0 and 1. If all samples in data set, S, belong to one class, then entropy will equal zero. If half of the samples are classified as one class and the other half are in another class, entropy will be at its highest at 1.In order to select the best feature to split on and find the optimal decision tree, the attribute with the smallest amount of entropy should be used. Information gain represents the difference in entropy before and after a split on a given attribute. The attribute with the highest information gain will produce the best split as it's doing the best job at classifying the training data according to its target classification.

Information gain is usually represented with the following equation,

where:

$$Information\ Gain\ (S, a) = \ Entropy(S) \ - \sum_{v \in values(a)} \frac{|S_v|}{|S|} . Entropy(S_v)$$

1. a represents a specific attribute or class label.
2. $Entropy(S)$ is the entropy of dataset, *S*.
3. $\frac{|S_v|}{|S|}$ is represents the proportion of the values in S_v to the number of values in the dataset, *S*.
4. $Entropy(S_v)$ is the entropy of dataset, S_v.

Gini Impurity:

Gini impurity is the probability of incorrectly classifying random data point in the dataset if it were labeled based on the class distribution of the dataset. Similar to entropy, if set, S, is pure—i.e. belonging to one class) then, its impurity is zero. This is defined by the following equation :

$$Gini\ Impurity = 1 - \sum_{i} (p_i)^2$$

1.2 Metrics used for Model Evaluation:

I have used following metrics for all evaluation models including the meta model.

1)**Accuracy:**

Accuracy simply measures how often the classifier correctly predicts. We can define accuracy as the ratio of the number of correct predictions and the total number of predictions**.**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where, TP = True Positive

TN = True Negative

FP = False Positive & FN = False Negative.

2**) Precision:**

Precision for a label is defined as the number of true positives divided by the number of predicted positives. It explains how many of the correctly predicted cases actually turned out to be positive. Precision is useful in the cases where False Positive is a higher concern than False Negatives.

$$Precision = \frac{TP}{TP + FP}$$

3**) Recall:**

Recall for a label is defined as the number of true positives divided by the total number of actual positives. It explains how many of the actual positive cases we were able to predict correctly with our model. Recall is a useful metric in cases where False Negative is of higher concern than False Positive.

$$Recall = \frac{TP}{TP + FN}$$

4) **F1 Score:**

F1 Score is the harmonic mean of precision and recall. It gives a combined idea about Precision and Recall metrics. It is maximum when Precision is equal to Recall.

$$F1 = 2.\frac{Precision * Recall}{Precision + Recall}$$

The F1 score punishes extreme values more.

5) **Confusion Matrix:**

A confusion matrix is defined as a table that is often used to describe the performance of a classification model on a set of the test data for which the true values are known. It is a table with combinations of predicted and actual values.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

6) **AUC-ROC:**

The **Receiver Operator Characteristic (ROC)** is a probability curve that plots the TPR(True Positive Rate) against the FPR(False Positive Rate) at various threshold values and separates the 'signal' from the 'noise'.

The **Area Under the Curve (AUC)** is the measure of the ability of a classifier to distinguish between classes. From the graph, we simply say the area of the curve ABDE and the X and Y-axis. the greater the AUC, the better is the performance of the model at different threshold points between positive and negative classes.

1.3 Evaluation Of Decision Tree Classifier(with default hyperparameters):

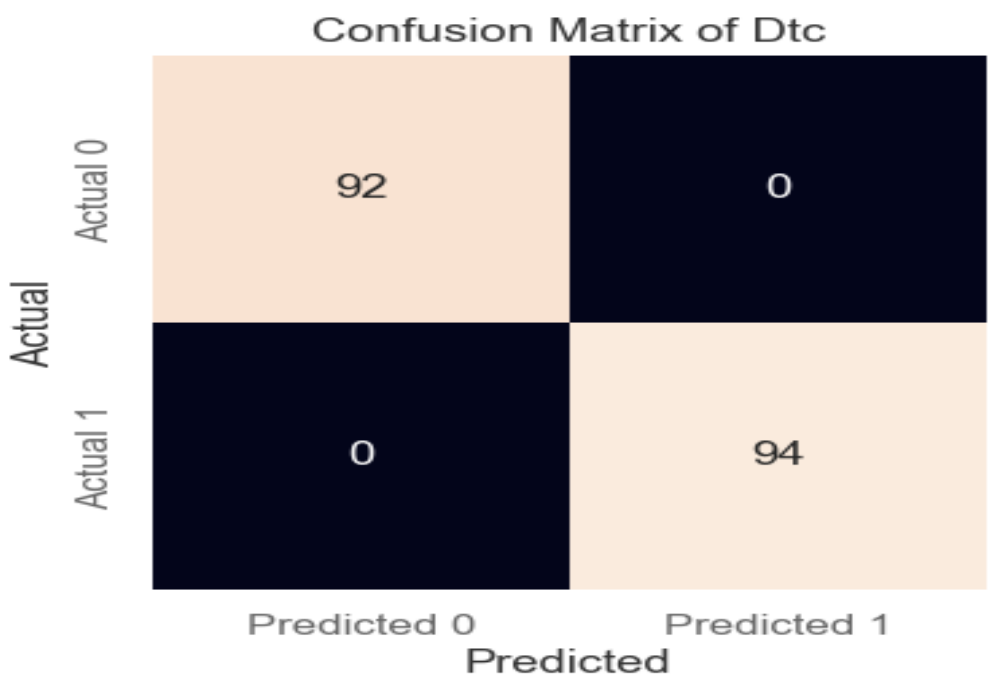
I have implemented the Decision Trees Classifier along with default hyperparameters., initially.

```
Area under ROC Curve of dtc: 1.0
Confusion Matrix of dtc:
[[92  0]
 [ 0 94]]
Classification Report of dtc:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        92
           1       1.00      1.00      1.00        94

    accuracy                           1.00       186
   macro avg       1.00      1.00      1.00       186
weighted avg       1.00      1.00      1.00       186

Correct Prediction of dtc %:99.46524064171123
```



2. Random Forest Classifier:

Random Forest Algorithm widespread popularity stems from its user-friendly nature and adaptability, enabling it to tackle both classification and regression problems effectively. The algorithm's strength lies in its ability to handle complex datasets and mitigate overfitting, making it a valuable tool for various predictive tasks in machine learning.

One of the most important features of the Random Forest Algorithm is that it can handle the data set containing ***continuous variables,*** as in the case of regression, and ***categorical variables,*** as in the case of classification. It performs better for classification and regression tasks. In this tutorial, we will understand the working of random forest and implement random forest on a classification task.

Before understanding the working of the random forest algorithm in machine learning, we must look into the ensemble learning technique.

Ensemble simply means combining multiple models. Thus a collection of models is used to make predictions rather than an individual model.

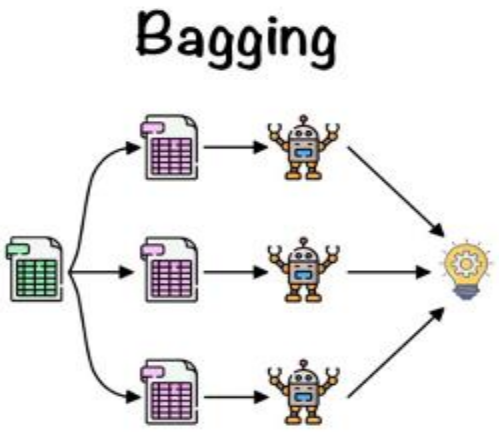
Ensemble uses two types of methods:

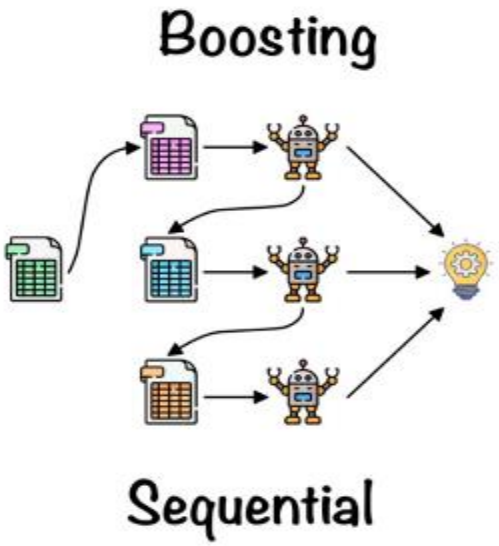
Bagging:

It creates a different training subset from sample training data with replacement & the final output is based on majority voting. For example, Random Forest.

Boosting:

It combines weak learners into strong learners by creating sequential models such that the final model has the highest accuracy. For example, ADABoost, XGBoost.





Bagging, also known as Bootstrap Aggregation, serves as the ensemble technique in the Random Forest algorithm. Here are the steps involved in Bagging:

1. **Selection of Subset**: Bagging starts by choosing a random sample, or subset, from the entire dataset.
2. **Bootstrap Sampling**: Each model is then created from these samples, called Bootstrap Samples, which are taken from the original data with replacement. This process is known as row sampling.
3. **Bootstrapping**: The step of row sampling with replacement is referred to as bootstrapping.
4. **Independent Model Training**: Each model is trained independently on its corresponding Bootstrap Sample. This training process generates results for each model.
5. **Majority Voting**: The final output is determined by combining the results of all models through majority voting. The most commonly predicted outcome among the models is selected.
6. **Aggregation**: This step, which involves combining all the results and generating the final output based on majority voting, is known as aggregation.

Steps Involved in Random Forest Algorithm:

- **Step 1:** In the Random forest model, a subset of data points and a subset of features is selected for constructing each decision tree. Simply put, n random records and m features are taken from the data set having k number of records.
- **Step 2:** Individual decision trees are constructed for each sample.
- **Step 3:** Each decision tree will generate an output.
- **Step 4:** Final output is considered based on ***Majority Voting or Averaging*** for Classification and regression, respectively.

Difference Between Decision Tree and Random Forest:

Random forest is a collection of decision trees; still, there are a lot of differences in their behavior.

Decision trees	Random Forest
1. Decision trees normally suffer from the problem of overfitting if it's allowed to grow without any control.	1. Random forests are created from subsets of data, and the final output is based on average or majority ranking; hence the problem of overfitting is taken care of.
2. A single decision tree is faster in computation.	2. It is comparatively slower.
3. When a data set with features is taken as input by a decision tree, it will formulate some rules to make predictions.	3. Random forest randomly selects observations, builds a decision tree, and takes the average result. It doesn't use any set of formulas.

Important Hyperparameters in Random Forest:

- **n_estimators:** Number of trees the algorithm builds before averaging the predictions**.**
- **max_features:** Maximum number of features random forest considers splitting a node.
- **mini_sample_leaf:** Determines the minimum number of leaves required to split an internal node**.**
- **criterion:** How to split the node in each tree? (Entropy/Gini impurity/Log Loss).
- **max_leaf_nodes:** Maximum leaf nodes in each tree.
- **n_jobs:** it tells the engine how many processors it is allowed to use. If the value is 1, it can use only one processor, but if the value is -1, there is no limit.
- **random_state:** controls randomness of the sample. The model will always produce the same results if it has a definite value of random state and has been given the same hyperparameters and training data**.**
- **oob_score:** OOB means out of the bag. It is a random forest cross-validation method. In this, one-third of the sample is not used to train the data; instead used to evaluate its performance. These samples are called out-of-bag samples.

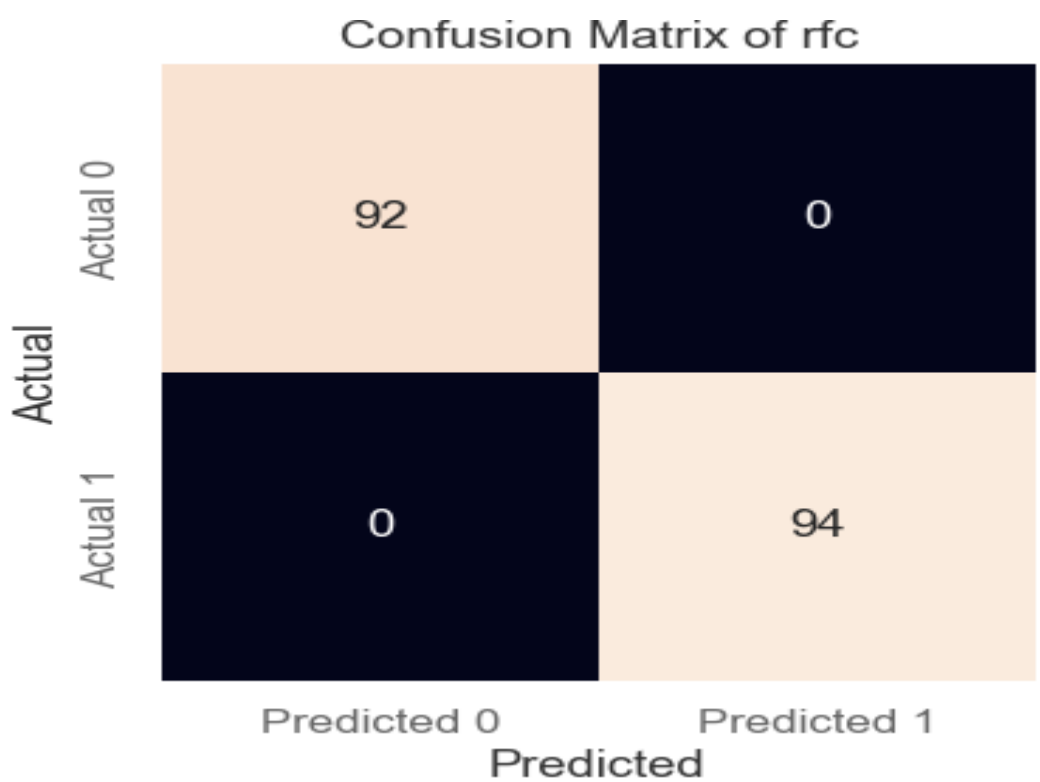
2.1 Evaluation of Random forest (with default hyperparameters):

```
Area under ROC Curve of rfc: 1.0
Confusion Matrix of rfc:
[[92  0]
 [ 0 94]]
Classification Report of rfc:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        92
           1       1.00      1.00      1.00        94

    accuracy                           1.00       186
   macro avg       1.00      1.00      1.00       186
weighted avg       1.00      1.00      1.00       186

Correct Prediction of rfc %:99.46524064171123
```



3. KNN Classifier:

The k-nearest neighbors algorithm, also known as KNN or k-NN, is a non-parametric, supervised learning classifier, which uses proximity to make classifications or predictions about the grouping of an individual data point. While it can be used for either regression or classification problems, it is typically used as a classification algorithm, working off the assumption that similar points can be found near one another**.**

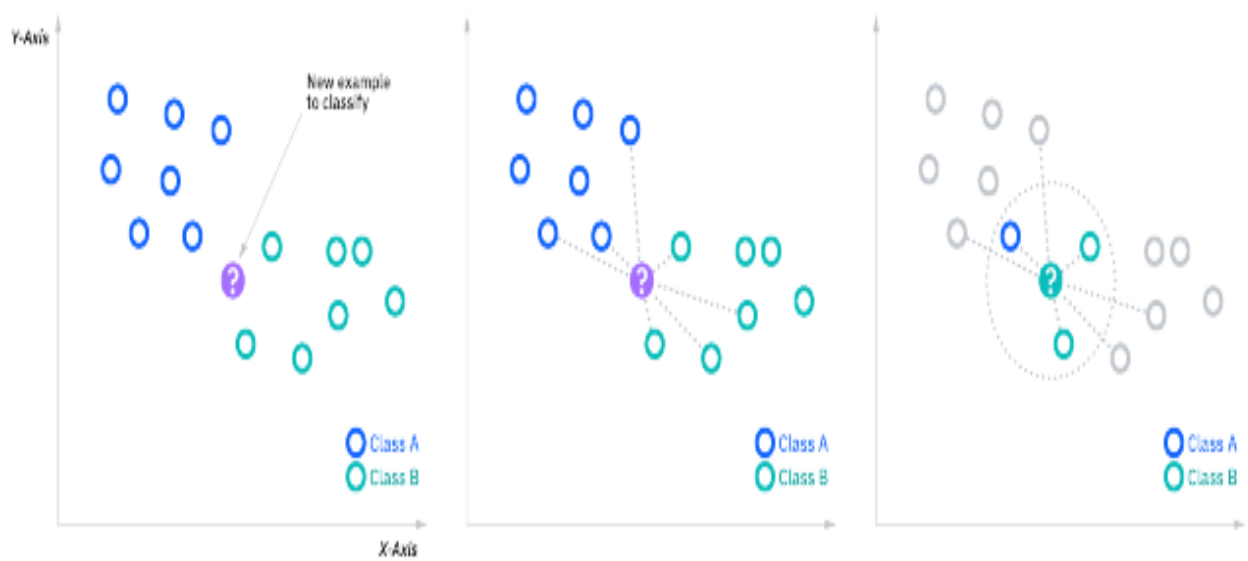
For classification problems, a class label is assigned on the basis of a majority vote—i.e. the label that is most frequently represented around a given data point is used. While this is technically considered “plurality voting”, the term, “majority vote” is more commonly used in literature. The distinction between these terminologies is that “majority voting” technically requires a majority of greater than 50%, which primarily works when there are only two categories. When you have multiple classes—e.g. four categories, you don’t necessarily need 50% of the vote to make a conclusion about a class; you could assign a class label with a vote of greater than 25%.

The goal of the k-nearest neighbor algorithm is to identify the nearest neighbors of a given query point, so that we can assign a class label to that point.

In order to determine which data points are closest to a given query point, the distance between the query point and the other data points will need to be calculated. This distance metric help to form decision boundaries, which partitions query points into different regions.

Euclidean distance is the most commonly used distance measure, and it is limited to real-valued vectors. Using the below formula, it measures a straight line between the query point and the other point being measured.

$$d(x, y) = \sum_{i=1}^{n} \sqrt{(x_i - y_i)^2}$$



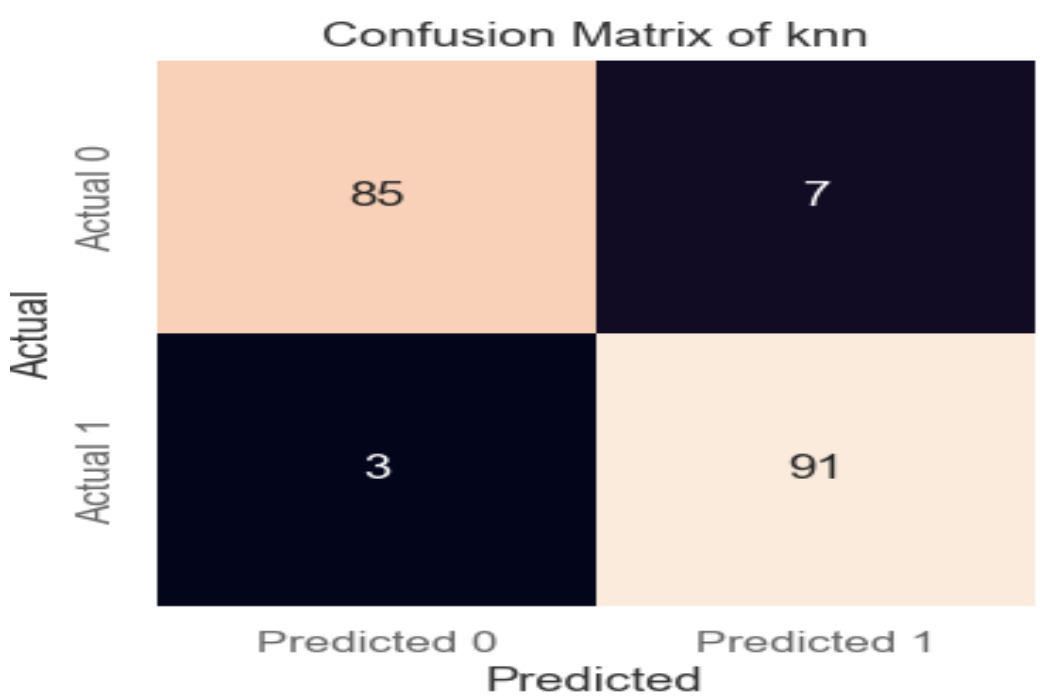
3.1 Evaluation of KNN Classifier (with default hyperparameters):

```
Area under ROC Curve of knn: 0.9459990749306197
Confusion Matrix of knn:
[[85  7]
 [ 3 91]]
Classification Report of knn:
              precision    recall  f1-score   support

           0       0.97      0.92      0.94        92
           1       0.93      0.97      0.95        94

    accuracy                           0.95       186
   macro avg       0.95      0.95      0.95       186
weighted avg       0.95      0.95      0.95       186

Correct Prediction of knn %:93.58288770053476
```



4. XGBoost Classifier:

XGBoost is a machine learning algorithm that belongs to the ensemble learning category, specifically the gradient boosting framework. It utilizes decision trees as base learners and employs regularization techniques to enhance model generalization. Known for its computational efficiency, feature importance analysis, and handling of missing values, XGBoost is widely used for tasks such as regression, classification, and ranking.

XGBoost is an ensemble learning method. Sometimes, it may not be sufficient to rely upon the results of just one machine learning model. Ensemble learning offers a systematic solution to combine the predictive power of multiple learners. The resultant is a single model which gives the aggregated output from several models. The models that form the ensemble, also known as base learners, could be either from the same learning algorithm or different learning algorithms. We know that Bagging and boosting are two widely used ensemble learners. Though these two techniques can be used with several statistical models, the most predominant usage has been with decision trees.

In boosting, the trees are built sequentially such that each subsequent tree aims to reduce the errors of the previous tree. Each tree learns from its predecessors and updates the residual errors. Hence, the tree that grows next in the sequence will learn from an updated version of the residuals.

The base learners in boosting are weak learners in which the bias is high, and the predictive power is just a tad better than random guessing. Each of these weak learners contributes some vital information for prediction, enabling the boosting technique to produce a strong learner by effectively combining these weak learners. The final strong learner brings down both the bias and the variance.

In contrast to bagging techniques like Random Forest, in which trees are grown to their maximum extent, boosting makes use of trees with fewer splits. Such small trees, which are not very deep, are highly interpretable. Parameters like the number of trees or iterations, the rate at which the gradient boosting learns, and the depth of the tree, could be optimally selected through validation techniques like k-fold cross validation. Having a large number of trees might lead to overfitting. So, it is necessary to carefully choose the stopping criteria for boosting.

The gradient boosting ensemble technique consists of three simple steps:

- An initial model F0 is defined to predict the target variable y. This model will be associated with a residual (y – F0)
- A new model h1 is fit to the residuals from the previous step
- Now, F0 and h1 are combined to give F1, the boosted version of F0. The mean squared error from F1 will be lower than that from F0:

  $F1(x) < -F0(x) + h1(x)$

To improve the performance of F1, we could model after the residuals of F1 and create a new model F2:

F2(x) < -F1(x) + h2(x)

This can be done for *'m'* iterations, until residuals have been minimized as much as possible:

Fm(x) < -Fm-1(x) + hm(x)

Here, the additive learners do not disturb the functions created in the previous steps. Instead, they impart information of their own to bring down the errors.

In the process, the model should be initialized with a function F0(x). F0(x) should be a function which minimizes the loss function or MSE (mean squared error), in this case:

- F0(x) – with which we initialize the boosting algorithm – is to be defined:

$$F_0(x) = argmin_\gamma \sum_{i=1}^{n} L(y_i, \gamma)$$

$$argmin_\gamma \sum_{i=1}^{n} L(y_i, \gamma) = argmin_\gamma \sum_{i=1}^{n} (y_i - \gamma)^2$$

- The gradient of the loss function is computed iteratively:

$$r_{im} = -\alpha \left[\frac{\partial(L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}, \text{ where } \alpha \text{ is the learning rate}$$

- Each hm(x) is fit on the gradient obtained at each step.
- The multiplicative factor γm for each terminal node is derived and the boosted model Fm(x) is defined:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

4.1 Unique Features of XGBoost:

XGBoost is a popular implementation of gradient boosting. Let's discuss some features of XGBoost that make it so interesting:

- **Regularization:** XGBoost has an option to penalize complex models through both L1 and L2 regularization. Regularization helps in preventing overfitting
- **Handling sparse data:** Missing values or data processing steps like one-hot encoding make data sparse. XGBoost incorporates a sparsity-aware split finding algorithm to handle different types of sparsity patterns in the data
- **Weighted quantile sketch:** Most existing tree based algorithms can find the split points when the data points are of equal weights (using quantile sketch algorithm). However, they are not equipped to handle weighted data. XGBoost has a distributed weighted quantile sketch algorithm to effectively handle weighted data

- **Block structure for parallel learning**: For faster computing, XGBoost can make use of multiple cores on the CPU. This is possible because of a block structure in its system design. Data is sorted and stored in in-memory units called blocks. Unlike other algorithms, this enables the data layout to be reused by subsequent iterations, instead of computing it again. This feature also serves useful for steps like split finding and column sub-sampling
- **Cache awareness**: In XGBoost, non-continuous memory access is required to get the gradient statistics by row index. Hence, XGBoost has been designed to make optimal use of hardware. This is done by allocating internal buffers in each thread, where the gradient statistics can be stored
- **Out-of-core computing**: This feature optimizes the available disk space and maximizes its usage when handling huge datasets that do not fit into memory

4.2 XGBoost Hyperparameters:

The overall parameters have been divided into 3 categories by XGBoost authors:

1. **General Parameters:** Guide the overall functioning
2. **Booster Parameters:** Guide the individual booster (tree/regression) at each step
3. **Learning Task Parameters:** Guide the optimization performed

4.2.1 General Parameters:

These define the overall functionality of XGBoost.

1. **booster [default=gbtree]**
    - Select the type of model to run at each iteration. It has 2 options:
        - gbtree: tree-based models
        - gblinear: linear models
2. **silent [default=0]**
    - Silent mode is activated is set to 1, i.e., no running messages will be printed.

- It's generally good to keep it 0 as the messages might help in understanding the model.

3. **nthread [default to the maximum number of threads available if not set]**
   - This is used for parallel processing, and the number of cores in the system should be entered
   - if you wish to run on all cores, the value should not be entered, and the algorithm will detect it automatically.

4.2.2 Boosting Parameters:

Parameter	Default	Description
learning_rate	0.3	Shrink the weights on each step
n_estimators	100	Number of trees to fit.
objective	binary: logistic	logistic regression for binary classification
booster	gbtree	Select the model for each iteration
nthread	max	Input the system core number
min_child_weight	1	Minimum sum of weights
max_depth	6	Maximum depth of a tree.
gamma	0	The minimum loss reduction needed for splitting
subsample	1	Control the sample's proportion
colsample_bytree	1	Column's fraction of random samples
reg_lambda	1	L2 regularization term on weights
reg_alpha	0	L1 regularization term on weights

4.2.3 Learning Task Parameters:

These parameters are used to define the optimization objective and the metric to be calculated at each step.

1. **objective [default=reg:linear]**
    - This defines the loss function to be minimized. Mostly used values are:
        - **binary: logistic** –logistic regression for binary classification returns predicted probability (not class)
        - **multi: softmax** –multiclass classification using the softmax objective, returns predicted class (not probabilities)
            - you also need to set an additional **num_class** (number of classes) parameter defining the number of unique classes
        - **multi: softprob** –same as softmax, but returns predicted probability of each data point belonging to each class.
2. **eval_metric [default according to objective]**
    - The evaluation metrics are to be used for validation data.
    - The default values are rmse for regression and error for classification.
    - Typical values are:
        - **rmse** – root mean square error

- **mae** – mean absolute error
- **logloss** – negative log-likelihood
- **error** – Binary classification error rate (0.5 thresholds)
- **merror** – Multiclass classification error rate
- **mlogloss** – Multiclass logloss
- **auc:** Area under the curve

3. **seed [default=0]**
    - The random number seed.
    - It can be used for generating reproducible results and also for parameter tuning.

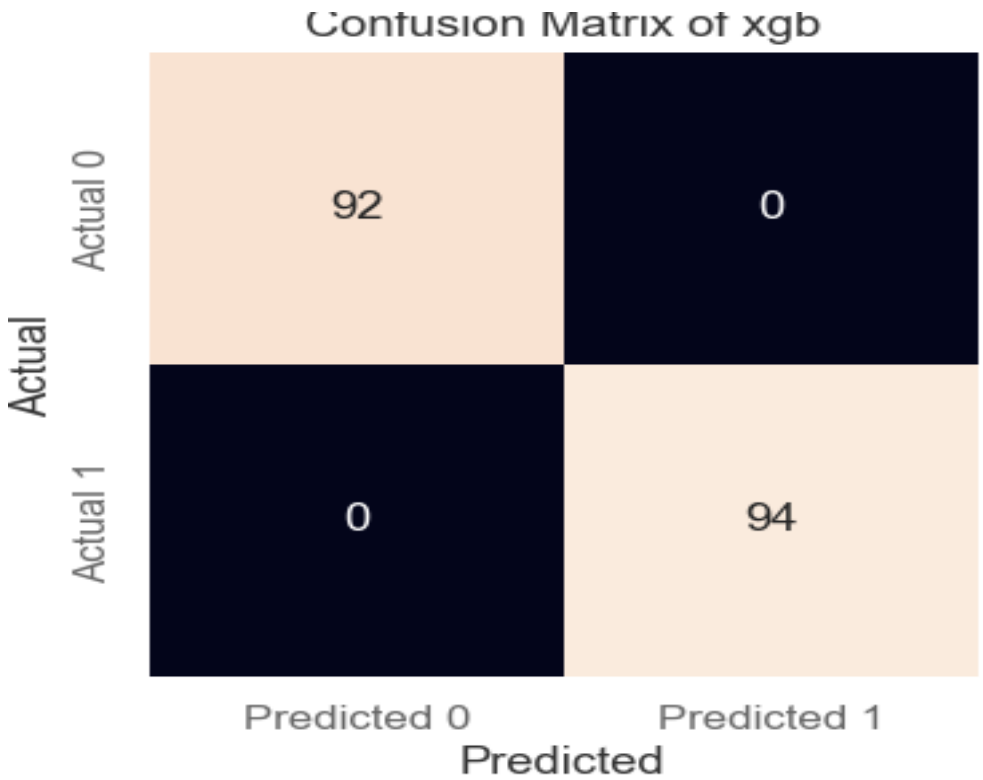
4.3 Evaluation of XGBoost (with default parameters):

```
Area under ROC Curve of xgb: 1.0
Confusion Matrix of xgb:
[[92  0]
 [ 0 94]]
Classification Report of xgb:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        92
           1       1.00      1.00      1.00        94

    accuracy                           1.00       186
   macro avg       1.00      1.00      1.00       186
weighted avg       1.00      1.00      1.00       186

Correct Prediction of xgb %:100.0
```

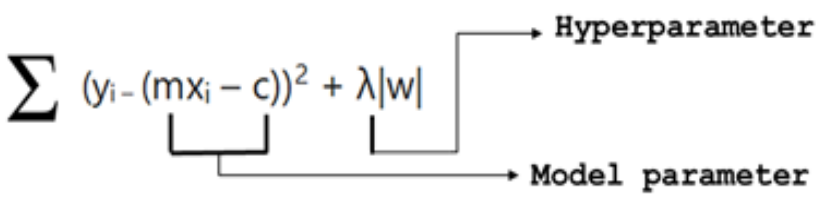


IV. Hyperparameter Tuning

As we know that there are parameters that are internally learned from the given dataset and derived from the dataset, they are represented in making predictions, classification and etc., These are so-called **Model Parameters,** and they are varying with respect to the nature of the data we couldn't control this since it depends on the data. Like '**m**' and '**C**' in linear equation, which is the value of coefficients learned from the given dataset.

Some set of parameters that are used to control the behaviour of the model/algorithm and adjustable in order to obtain an improvised model with optimal performance is so-called **Hyperparameters.**

For better clarity and understanding, here is one more classical representation,



$$\sum (y_i - (mx_i - c))^2 + \lambda|w|$$

Hyperparameters are supplied as arguments to the model algorithm during initializing them as key, value and their values are picked by the data scientist, who is building the model in iterative mode.

As we know that there is a list of HPs for any selected algorithm(s) and our job is to figure out the best combination of HPs and to get the optimal results by tweaking them strategically, this process will be providing us with the platform for **Hyperparameter Space** and this combination leads to provide the best optimal results, no doubt in that but finding this combo is not so easy, we have to search throughout the space. Here every combination of selected HP value is said to be the "**MODEL**" and have to evaluate the same on the spot. For this reason, there are two generic approaches to search effectively in the HP space

are **GridSearch CV** and **RandomSearch CV.** Here CV denotes **Cross-Validation.**

I have tunned the hyperparameters of all four base models which are as follows:

```
Best Hyperparameters for Decision Tree: {'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2}
Best Hyperparameters for Random Forest: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 50}
Best Hyperparameters for K Neighbors: {'n_neighbors': 7, 'p': 1, 'weights': 'distance'}
Best Hyperparameters for XGBoost: {'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50, 'subsample': 0.8}
```

V. Meta Model/ Blended Ensemble:

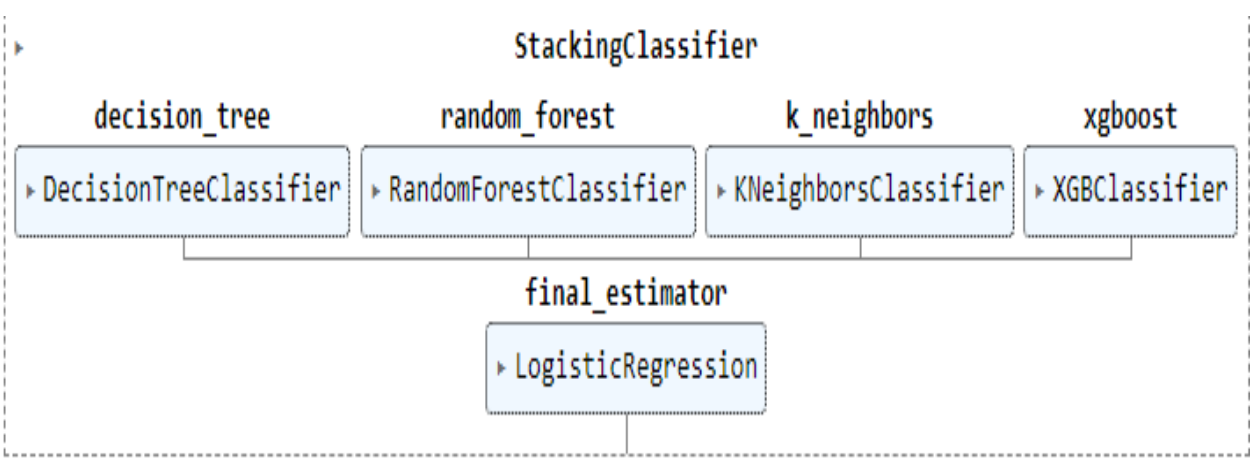
1. Construction:

For this study, I have stacked all four base classifiers (Decision Trees, Random Forest, KNN, XGBoost) with tunned/optimized hyperparameters (of each model) into a stacked classifier which works an ensemble with Logistic regression as a final estimator.

```
# Create a stacked classifier with Logistic Regression as the final estimator

stacked_classifier = StackingClassifier(
    estimators=[
        ('decision_tree', grid_search_dt.best_estimator_),
        ('random_forest', grid_search_rf.best_estimator_),
        ('k_neighbors', grid_search_knn.best_estimator_),
        ('xgboost', grid_search_xgb.best_estimator_)
    ],
    final_estimator=LogisticRegression()
)

# Fit the stacked classifier
stacked_classifier.fit(X_train, y_train)
```



2. Meta Model Evaluation:

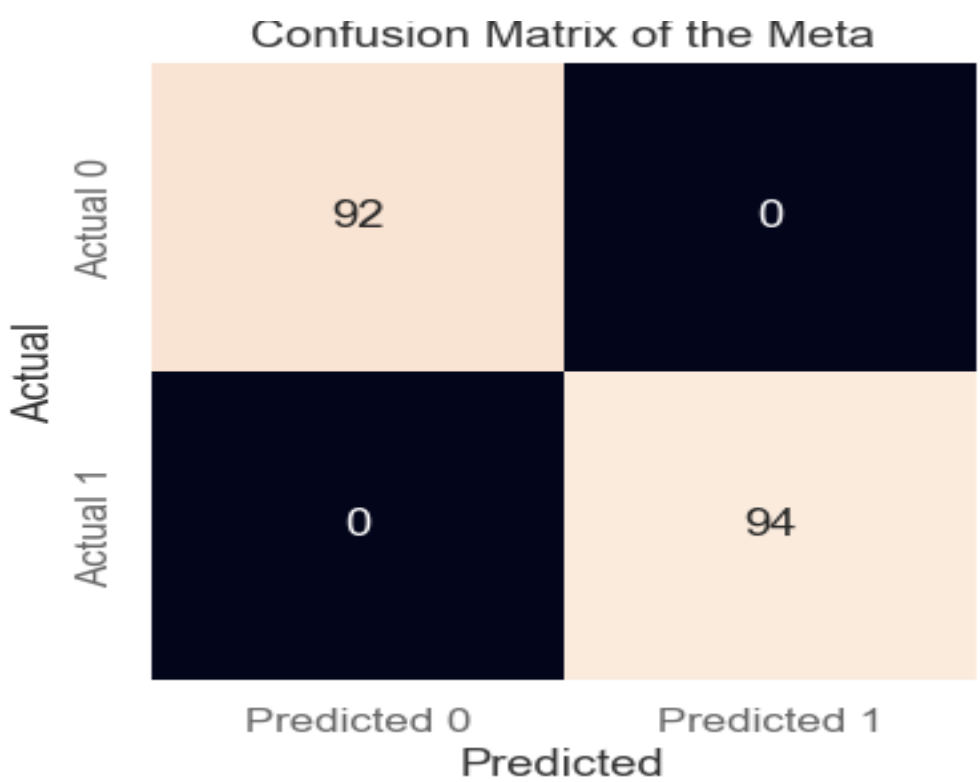
2.1 Meta Model Evaluation Metrics:

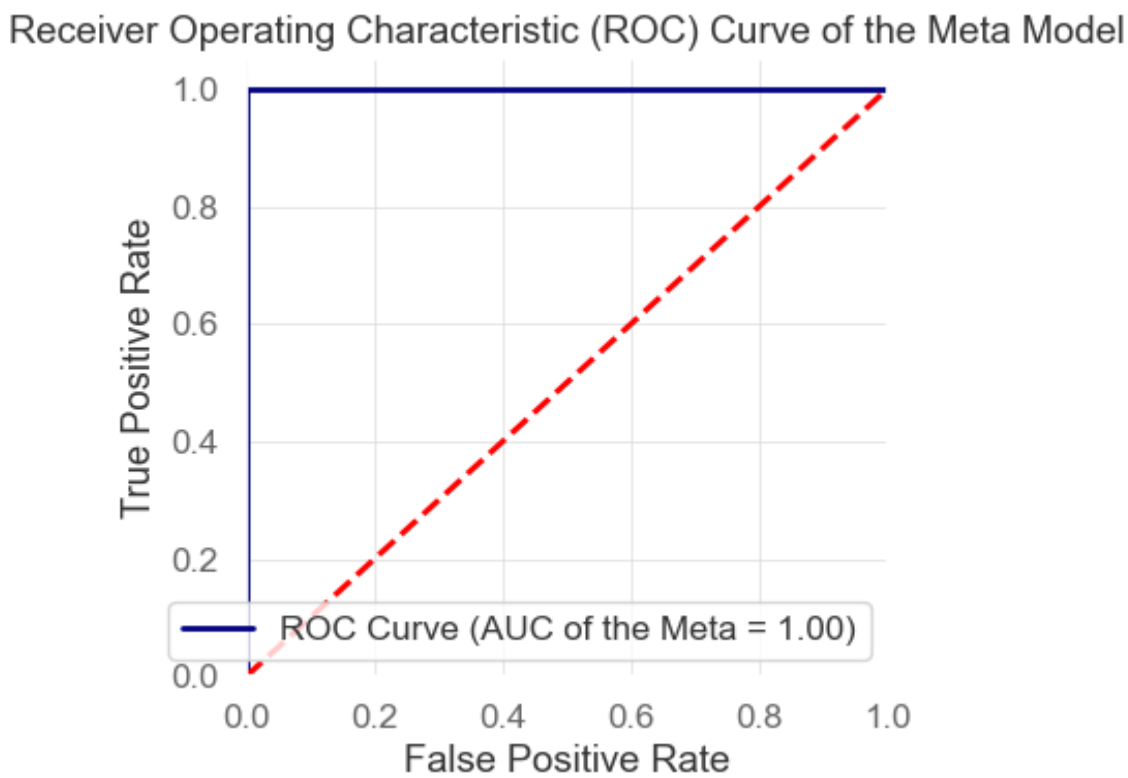
```
Area under ROC Curve of the Meta: 1.0
Confusion Matrix of the Meta:
[[92  0]
 [ 0 94]]
Classification Report of the Meta:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        92
           1       1.00      1.00      1.00        94

    accuracy                           1.00       186
   macro avg       1.00      1.00      1.00       186
weighted avg       1.00      1.00      1.00       186

Correct Prediction of the Meta %:99.46524064171123
```





2.2 Application to a Trading Strategy :

I have applied the meta model to the testing dataset for a trading strategy.

```
# Now,We will visualize the strategy returns in the histogram plot.

df['Strategy_Returns'] = df['Daily_Returns'].shift(-1) * stacked_classifier.predict(X)

# As '870' is size of trainig dataset we want to evaluate the model on the testing dataset

df.Strategy_Returns.iloc[870:].hist(figsize=(4,4))
plt.title('Strategy Returns Histogram')
plt.ylabel('Percentage %')
plt.show()
```

2.2.1 Strategy Performance Results & Metrics :

I have used ‘QuantStats’ Library, to evaluate the strategy.

Cumulative Return	47.59%
CAGR%	19.96%
Sharpe	0.86
Prob. Sharpe Ratio	85.04%
Smart Sharpe	0.83
Sortino	1.28
Smart Sortino	1.23
Sortino/√2	0.9
Smart Sortino/√2	0.87
Omega	1.22

VI. Conclusion

This study is sought to perform an extensive analysis of ensemble methods such as bagging, boosting, stacking, and blending for stock-market prediction, using stock market dataset of 'TSLA' over the time horizon of five years. Four base classifiers, Decision Trees, Random Forest, KNN, XGBoost have been used to form a stacked ensemble. Ensemble modeling can exponentially boost the performance of your model and can sometimes be the deciding factor between first place and second !. In this study, we have covered various ensemble learning techniques and saw how these techniques are applied in stock trend prediction using Machine Learning algorithms.

VII. References

- https://scikit-learn.org/stable/modules/tree.html
- https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
- https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html
- https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html
- CQF Final Project Tutorial II, Kannan Singaravelu,CQF
- "A comprehensive evaluation of ensemble learning for stock-market prediction" , by Isaac Kofi Nti, Adebayo Felix Adekoya & Benjamin Asubam Weyori ,published in 'Journal of Big Data' volume 7, Article number: 20 (2020). https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00299-5#Bib1